

- » Discovering the basics of DNS
- » Exploring zones
- » Examining resource records
- » Configuring a DNS server
- » Setting up a DNS client

Chapter 6

DNS

Domain Name System (DNS) is the Transmission Control Protocol/Internet Protocol (TCP/IP) facility that lets you use names rather than numbers to refer to host computers. Without DNS, you'd buy books from 54.239.28.85 instead of from `www.amazon.com` and you'd sell your used furniture at 23.216.149.153 instead of on `www.ebay.com`.

Understanding how DNS works and how to set up a DNS server is crucial to setting up and administering a TCP/IP network. (For more on TCP/IP, see Book 2, Chapter 2.) This chapter introduces you to the basics of DNS, including how the DNS naming system works and how to set up a DNS server.



TECHNICAL
STUFF

If you want to review the complete official specifications for DNS, look up RFC 1034 and 1035 at www.ietf.org/rfc/rfc1034.txt and www.ietf.org/rfc/rfc1035.txt, respectively.

Understanding DNS Names

DNS is a name service that provides a standardized system for providing names to identify TCP/IP hosts as well as a way to look up the IP address of a host, given the host's DNS name. For example, if you use DNS to look up the name `www.ebay.com`, you get the IP address of the eBay web host: 23.216.149.153. Thus, DNS allows you to access the eBay website by using the DNS name `www.ebay.com` instead of the site's IP address.



Note that by the time you get this book, read this chapter, and try to actually look up the IP address for `www.ebay.com`, there's a very good chance you'll discover that eBay has changed its IP address.

The following sections describe the basic concepts of DNS.

Domains and domain names

To provide a unique DNS name for every host computer on the internet, DNS uses a time-tested technique: Divide and conquer. DNS uses a hierarchical naming system that's similar to how folders are organized hierarchically on a Windows computer. Instead of folders, however, DNS organizes its names into domains. Each domain includes all the names that appear directly beneath it in the DNS hierarchy.

For example, Figure 6-1 shows a small portion of the DNS domain tree. At the very top of the tree is the *root domain*, which is the anchor point for all domains. Directly beneath the root domain are four top-level domains, named `edu`, `com`, `org`, and `gov`.

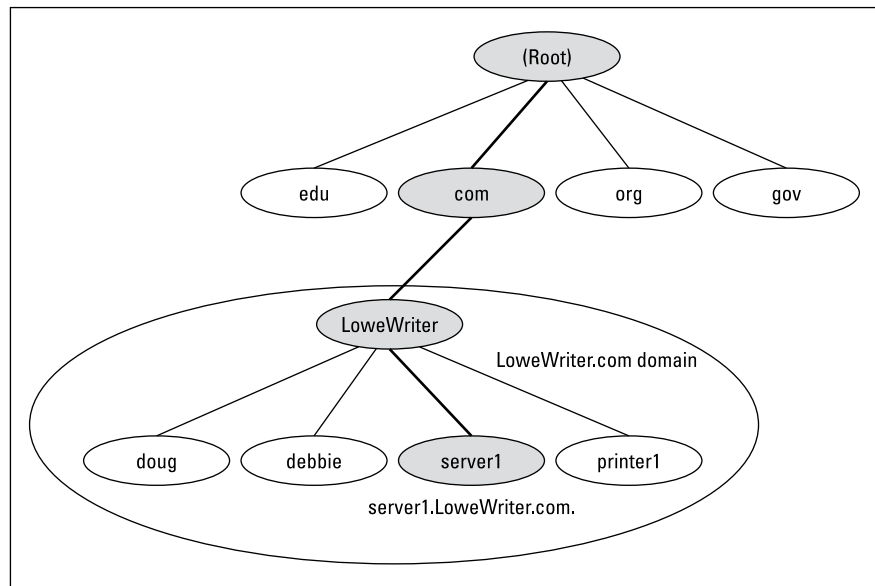


FIGURE 6-1:
DNS names.

In reality, many more top-level domains than this exist in the internet's root domain. For more information, see the section "Top-Level Domains," later in this chapter.

Beneath the `com` domain in Figure 6-1 is another domain called `LoweWriter`, which happens to be my own personal domain. (Pretty clever, eh?) To completely identify this domain, you have to combine it with the name of its *parent domain* (in this case, `com`) to create the complete domain name: `LoweWriter.com`. Notice that the parts of the domain name are separated from each other with periods, which are called dots. As a result, when you read this domain name, you pronounce it *LoweWriter dot com*.

Beneath the `LoweWriter` node are four host nodes, named `doug`, `debbie`, `server1`, and `printer1`. Respectively, these correspond to three computers and a printer on my home network. You can combine the host name with the domain name to get the complete DNS name for each of my network's hosts. For example, the complete DNS name for my server is `server1.LoweWriter.com`. Likewise, my printer is `printer1.LoweWriter.com`.

Here are a few additional details that you need to remember about DNS names:



WARNING

» **DNS names are not case sensitive.** As a result, `LoweWriter` and `Lowewriter` are treated as the same name, as are `LOWEWRIter`, `LOWewriter`, and `Lowewriter`. When you use a domain name, you can use capitalization to make the name easier to read, but DNS ignores the difference between capital and lowercase letters.

» **The name of each DNS node can be up to 63 characters long (not including the dot) and can include letters, numbers, and hyphens.**

No other special characters are allowed.

» **A *subdomain* is a domain that's beneath an existing domain.** For example, the `com` domain is actually a subdomain of the root domain. Likewise, `LoweWriter` is a subdomain of the `com` domain.

» **DNS is a hierarchical naming system that's similar to the hierarchical folder system used by Windows.**

However, one crucial difference exists between DNS and the Windows naming convention. When you construct a complete DNS name, you start at the bottom of the tree and work your way up to the root. Thus, `doug` is the lowest node in the name `doug.LoweWriter.com`. In contrast, Windows paths are the opposite: They start at the root and work their way down. For example, in the path `\Windows\System32\dns`, `dns` is the lowest node.

» **The DNS tree can be up to 127 levels deep.** However, in practice, the DNS tree is pretty shallow. Most DNS names have just three levels (not counting the root). And although you'll sometimes see names with four or five levels, you'll rarely see more levels than that.



TIP

» **Although the DNS tree is shallow, it's very broad.** In other words, each of the top-level domains has a huge number of second-level domains immediately beneath it. For example, at the time of this writing, the `com` domain had well over 100 million second-level domains beneath it.

Fully qualified domain names

If a domain name ends with a trailing dot, that trailing dot represents the root domain, and the domain name is said to be a *fully qualified domain name* (also known as an FQDN). A fully qualified domain name is also called an *absolute name*. A fully qualified domain name is unambiguous because it identifies itself all the way back to the root domain. In contrast, if a domain name doesn't end with a trailing dot, the name may be interpreted in the context of some other domain. Thus, DNS names that don't end with a trailing dot are called *relative names*.

This is similar to how relative and absolute paths work in Windows. For example, if a path begins with a backslash, such as `\Windows\System32\dns`, the path is absolute. However, a path that doesn't begin with a backslash, such as `System32\dns`, uses the current directory as its starting point. If the current directory happens to be `\Windows`, then `\Windows\System32\dns` and `System32\dns` refer to the same location.

In many cases, relative and fully qualified domain names are interchangeable because the software that interprets them always interprets relative names in the context of the root domain. That's why, for example, you can type `www.wiley.com` (without the trailing dot) — not `www.wiley.com.` — to go to the Wiley home page in a web browser. Some applications, such as DNS servers, may interpret relative names in the context of a domain other than the root.

DNS AND URLS

Whenever you use a web browser to navigate to a resource on the internet, you use a *Uniform Resource Locator* (URL) to find your way. But the format of a URL is much more complex than most people assume.

A URL can consist of five distinct parts, with the following syntax:

```
scheme://authority/path?query#fragment
```

The following paragraphs describe these five parts:

- **Scheme:** The *scheme* identifies the protocol used to access the resource. The scheme is followed by a colon (:). When working with HTTP, this should be (obviously) `http:`. Note that the scheme is used to determine the default port for TCP/IP. So, unless you explicitly specify a different port, using `http:` as the scheme will give you port 80.
- **Authority:** The *authority* portion of a URL is strangely named. It generally consists of a host name that can be resolved by DNS. But it can have two additional elements. The host name can be preceded by a username followed by the at symbol (@). And it can be followed by a port number, separated from the host by a colon (:). So, the authority might be `doug@lowewriter.com`, which looks like an email address but isn't. Or, it might be `www.lowewriter.com:8080`, indicating that instead of the port number suggested by the scheme, I want to use port 8080.
- **Path:** The *path* identifies a specific resource on the host server. It can be the name of a file or, more commonly, a file system path that lists one or more folders separated by slashes (/). The path always begins with a slash to separate it from the host name. For example: `files/java/examples.zip`.

A path may end with a filename, but that's not always necessary. If the filename is omitted, the HTTP server uses a default filename such as `index.html`.
- **Query:** The query part of a URL is optional but very useful. It provides additional information to the server. The query begins with a question mark (?) and consists of one or more key-value pairs in the form `key=value`. When you need more than one key-value pair in a query, separate the pairs with ampersands (&). For example, `?month=10&day=31` could be used to represent a date.
- **Fragment:** The fragment portion of a URL is used less commonly than the other elements but is still useful from time to time. It's usually used to refer to a specific part of the resource. For example, if the resource is an HTML file, the fragment may refer to a section within the file. For example, `#references` could be used to specify a references section within an HTML file.

Top-Level Domains

A *top-level domain* appears immediately beneath the root domain. Top-level domains come in two categories: generic domains and geographic domains. These categories are described in the following sections. (Actually, a third type of top-level domain exists, which is used for reverse lookups. I describe it later in this chapter, in the section “Reverse Lookup Zones.”)

Generic domains

Generic domains are the popular top-level domains that you see most often on the internet. Originally, seven top-level organizational domains existed. In 2002, seven more were added to help ease the congestion of the original seven — in particular, the `com` domain.

Table 6-1 summarizes the original seven generic top-level domains.

TABLE 6-1

The Original Seven Top-Level Domains

Domain	Description
<code>com</code>	Commercial organizations
<code>edu</code>	Educational institutions
<code>gov</code>	Government institutions
<code>int</code>	International treaty organizations
<code>mil</code>	Military institutions
<code>net</code>	Network providers
<code>org</code>	Noncommercial organizations

Because the `com` domain ballooned to an almost unmanageable size in the late 1990s, the internet authorities approved seven new top-level domains in an effort to take some of the heat off of the `com` domain. Most of these domains, listed in Table 6-2, became available in 2002.

TABLE 6-2

The New Seven Top-Level Domains

Domain	Description
<code>aero</code>	Aerospace industry
<code>biz</code>	Business
<code>coop</code>	Cooperatives
<code>info</code>	Informational sites
<code>museum</code>	Museums
<code>name</code>	Individual users
<code>pro</code>	Professional organizations

Country code domains

Although the top-level domains are open to anyone, U.S. companies and organizations dominate them. An additional set of top-level domains corresponds to international country code designations. Organizations outside the United States often use these top-level domains to avoid the congestion of the generic domains.

Table 6-3 lists those country code top-level domains with more than 200 registered subdomains at the time of this writing, in alphabetical order. In all, about 300 geographic top-level domains exist. The exact number varies from time to time as political circumstances change.

TABLE 6-3 **Country Code Top-Level Domains with More Than 200 Subdomains**

Domain	Description	Domain	Description
ac	Ascension Island	ie	Ireland
ae	United Arab Emirates	in	India
ag	Antigua and Barbuda	is	Iceland
am	Armenia	it	Italy
an	Netherlands Antilles	jp	Japan
as	American Samoa	kz	Kazakhstan
at	Austria	la	Lao People's Democratic Republic
be	Belgium	li	Liechtenstein
bg	Bulgaria	lk	Sri Lanka
br	Brazil	lt	Lithuania
by	Belarus	lu	Luxembourg
bz	Belize	lv	Latvia
ca	Canada	ma	Morocco
cc	Cocos (Keeling) Islands	md	Moldova
ch	Switzerland	nl	Netherlands
cl	Chile	no	Norway
cn	China	nu	Niue

(continued)

TABLE 6-3 *(continued)*

Domain	Description	Domain	Description
cx	Christmas Island	pl	Poland
cz	Czech Republic	pt	Portugal
de	Germany	ro	Romania
dk	Denmark	ru	Russian Federation
ee	Estonia	se	Sweden
es	Spain	si	Slovenia
eu	European Union	sk	Slovakia
fi	Finland	st	São Tomé and Príncipe
fm	Micronesia	su	Soviet Union
fo	Faroe Islands	to	Tonga
fr	France	tv	Tuvalu
ge	Georgia	tw	Taiwan
gr	Greece	ua	Ukraine
hr	Croatia	us	United States
hu	Hungary	ws	Samoa

The Hosts File

Long ago, in a network far, far away, the entire internet was small enough that network administrators could keep track of it all in a simple text file. This file, called the *Hosts file*, simply listed the name and IP address of every host on the network. Each computer had its own copy of the Hosts file. The trick was keeping all those Hosts files up to date. Whenever a new host was added to the internet, each network administrator would manually update his copy of the Hosts file to add the new host's name and IP address.

As the internet grew, so did the Hosts file. In the mid-1980s, it became obvious that a better solution was needed. Imagine trying to track the entire internet today by using a single text file to record the name and IP address of the millions of hosts on the internet! DNS was invented to solve this problem.

Understanding the Hosts file is important for two reasons:

- » **The Hosts file is not dead.** For small networks, a Hosts file may still be the easiest way to provide name resolution for the network's computers. In addition, a Hosts file can coexist with DNS. The Hosts file is always checked before DNS is used, so you can even use a Hosts file to override DNS if you want.
- » **The Hosts file is the precursor to DNS.** DNS was devised to circumvent the limitations of the Hosts file. You'll be in a better position to appreciate the benefits of DNS when you understand how the Hosts file works.

The Hosts file is a simple text file that contains lines that match IP addresses with host names. You can edit the Hosts file with any text editor, including Notepad. The exact location of the Hosts file depends on the client operating system, as listed in Table 6-4.

TABLE 6-4

Location of the Hosts File

Operating System	Location of Hosts File
Windows	c:\windows\system32\drivers\etc
Unix/Linux	/etc/hosts

All TCP/IP implementations are installed with a starter Hosts file. For example, Listing 6-1 shows a typical Windows TCP/IP Hosts file. As you can see, the starter file begins with some comments that explain the purpose of the file.

The Hosts file ends with comments, which show the host mapping commands used to map for the host name `localhost`, mapped to the IP address `127.0.0.1`. The IP address `127.0.0.1` is the standard loopback address. As a result, this entry allows a computer to refer to itself by using the name `localhost`.

Note that after the `127.0.0.1 localhost` entry, another `localhost` entry defines the standard IPv6 loopback address (`:::1`). This is required because all versions of Windows since Vista provide built-in support for IPv6.

Prior to Windows 7, these lines were not commented out in the Hosts file. But beginning with Windows 7, the name resolution for `localhost` is handled by DNS itself, so its definition isn't required in the Hosts file.

A Sample Hosts File

```
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each entry
# should be kept
# on an individual line. The IP address should be placed in the first column
# followed by
# the corresponding host name. The IP address and the host name should be
# separated by at
# least one space.
#
# Additionally, comments (such as these) may be inserted on individual lines or
# following
# the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97      rhino.acme.com      # source server
#      38.25.63.10      x.acme.com         # x client host

# localhost name resolution is handled within DNS itself.
#      127.0.0.1        localhost
#      ::1              localhost
```

To add an entry to the Hosts file, simply edit the file in any text editor. Then, add a line at the bottom of the file, after the `localhost` entry. Each line that you add should list the IP address and the host name that you want to use for the address. For example, to associate the host name `server1.LoweWriter.com` with the IP address `192.168.168.201`, you add this line to the Hosts file:

```
192.168.168.201 server1.LoweWriter.com
```

Then, whenever an application requests the IP address of the host name `server1.LoweWriter.com`, the IP address `192.168.168.201` is returned.

You can also add an alias to a host mapping. This enables users to access a host by using the alias as an alternative name. For example, consider the following line:

```
192.168.168.201 server1.LoweWriter.com s1
```

Here, the device at address `192.168.168.201` can be accessed as `server1.LoweWriter.com` or just `s1`.

Listing 6-2 shows a Hosts file with several hosts defined.

LISTING 6-2: A Hosts File with Several Hosts Defined

```
Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each entry
# should be kept
# on an individual line. The IP address should be placed in the first column
# followed by
# the corresponding host name. The IP address and the host name should be
# separated by at
# least one space.
#
# Additionally, comments (such as these) may be inserted on individual lines or
# following
# the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97      rhino.acme.com      # source server
#      38.25.63.10      x.acme.com          # x client host

# localhost name resolution is handled within DNS itself.
#      127.0.0.1        localhost
#      ::1              localhost
192.168.168.200 doug.LoweWriter.com #Doug's computer
192.168.168.201 server1.LoweWriter.com s1 #Main server
192.168.168.202 kristen.LoweWriter.com #Kristen's computer
192.168.168.203 printer1.LoweWriter.com p1 #HP Laser Printer
```

Note that even if your network uses DNS, every client still has a Hosts file.

Understanding DNS Servers and Zones

A *DNS server* is a computer that runs DNS server software, helps to maintain the DNS database, and responds to DNS name resolution requests from other computers. Although many DNS server implementations are available, the two most popular

are BIND and the Windows DNS service. BIND runs on Unix-based computers (including Linux computers), and Windows DNS (naturally) runs on Windows computers. Both provide essentially the same services and can interoperate.

The key to understanding how DNS servers work is to realize that the DNS database — that is, the list of all the domains, subdomains, and host mappings — is a massively distributed database. No single DNS server contains the entire DNS database. Instead, authority over different parts of the database is delegated to different servers throughout the internet.

For example, suppose that I set up a DNS server to handle name resolutions for my `LoweWriter.com` domain. Then, when someone requests the IP address of `doug.LoweWriter.com`, my DNS server can provide the answer. However, my DNS server wouldn't be responsible for the rest of the internet. Instead, if someone asks my DNS server for the IP address of some other computer, such as `coyote.acme.com`, my DNS server will have to pass the request on to another DNS server that knows the answer.

Zones

To simplify the management of the DNS database, the entire DNS namespace is divided into zones, and the responsibility for each zone is delegated to a particular DNS server. In many cases, zones correspond directly to domains. For example, if I set up a domain named `LoweWriter.com`, I can also set up a DNS zone called `LoweWriter.com` that's responsible for the entire `LoweWriter.com` domain.

However, the subdomains that make up a domain can be parceled out to separate zones, as shown in Figure 6-2. Here, a domain named `LoweWriter.com` has been divided into two zones. One zone, `us.LoweWriter.com`, is responsible for the entire `us.LoweWriter.com` subdomain. The other zone, `LoweWriter.com`, is responsible for the entire `LoweWriter.com` domain except for the `us.LoweWriter.com` subdomain.

Why would you do that? The main reason is to delegate authority for the zone to separate servers. For example, Figure 6-2 suggests that part of the `LoweWriter.com` domain is administered in the United States and that part of it is administered in France. The two zones in the figure allow one server to be completely responsible for the U.S. portion of the domain, and the other server handles the rest of the domain.

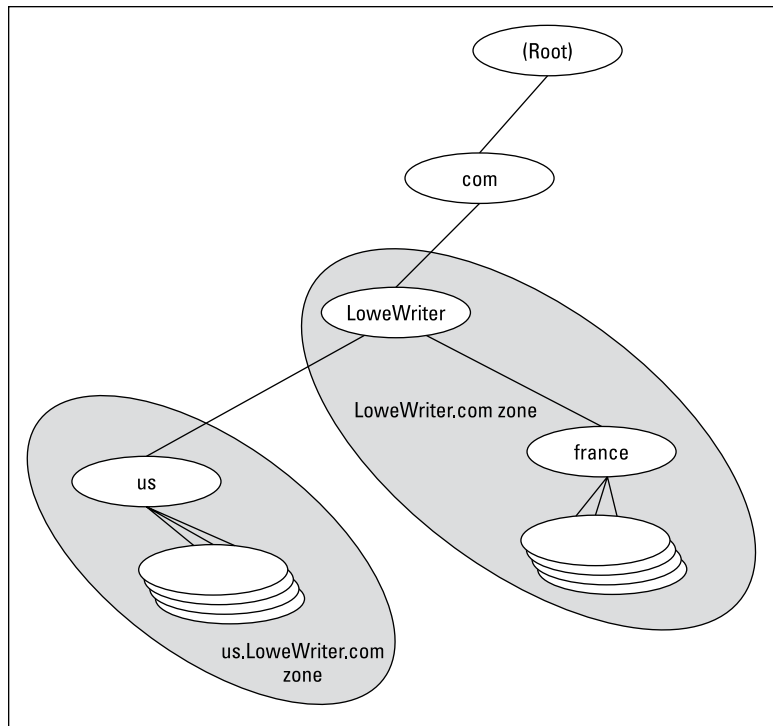


FIGURE 6-2:
DNS zones.

THE OLD PHONY HOSTS FILE TRICK

The Hosts file can be the basis of a fun, practical joke. Of course, neither I nor my editors or publishers recommend that you actually do this. If it gets you into trouble, don't send your lawyers to me. This sidebar is here only to let you know what to do if it happens to you.

The idea is to edit your poor victim's Hosts file so that whenever the user tries to access his favorite website, a site of your choosing comes up instead. For example, if you're trying to get your husband to take you on a cruise, add a line to his Hosts file that replaces his favorite website with the website for a cruise line. For example, this line should do the trick:

```
104.92.181.171 www.espn.com
```

Now, whenever your husband tries to call up the ESPN website, he'll get the Carnival Cruise Lines home page instead. (To find out the IP address of the website you want displayed, open a command prompt and type **ping** followed by a space and the URL of the

(continued)

(continued)

desired website. This will usually display the correct IP address. If not, you can use the `nslookup` command, as described in Book 2, Chapter 7.)

Of course, to actually pull a stunt like this would be completely irresponsible, especially if you didn't first make a backup copy of the Hosts file, just in case it somehow gets messed up.

Be warned: If the wrong websites suddenly start coming up, check your Hosts file to see whether it's been tampered with.

The following are the two basic types of zones:

- » A **primary zone** is the master copy of a zone. The data for a primary zone is stored in the local database of the DNS server that hosts the primary zone. Only one DNS server can host a particular primary zone. Any updates to the zone must be made to the primary zone.
- » A **secondary zone** is a read-only copy of a zone. When a server hosts a secondary zone, the server doesn't store a local copy of the zone data. Instead, it obtains its copy of the zone from the zone's primary server by using a process called *zone transfer*. Secondary servers must periodically check primary servers to see whether their secondary zone data is still current. If not, a zone transfer is initiated to update the secondary zone.

Primary and secondary servers

Each DNS server is responsible for one or more zones. The following are the two different roles that a DNS server can take:

- » **Primary server for a zone**, which means that the DNS server hosts a primary zone. The data for the zone is stored in files on the DNS server. Every zone must have one primary server.
- » **Secondary server for a zone**, which means that the DNS server obtains the data for a secondary zone from a primary server. Every zone should have at least one secondary server. That way, if the primary server goes down, the domain defined by the zone can be accessed via the secondary server or servers.



WARNING

A secondary server should be on a different subnet than the zone's primary server. If the primary and secondary servers are on the same subnet, both servers will be unavailable if the router that controls the subnet goes down.

Note that a single DNS server can be the primary server for some zones and a secondary server for other zones. A server is said to be *authoritative* for the primary and secondary zones that it hosts because it can provide definitive answers for queries against those zones.

Root servers

The core of DNS comprises the *root servers*, which are authoritative for the entire internet. The main function of the root servers is to provide the address of the DNS servers that are responsible for each of the top-level domains. These servers, in turn, can provide the DNS server address for subdomains beneath the top-level domains.

The root servers are a major part of the glue that holds the internet together. As you can imagine, they're swamped with requests day and night. A total of 13 root servers are located throughout the world. Table 6-5 lists the IP address and the organization that oversees the operation of each of the 13 root servers.

TABLE 6-5 **The 13 Root Servers**

Server	IP Address	Operator
A	198.41.0.4	VeriSign Global Registry Services
B	170.247.170.2	University of Southern California
C	192.33.4.12	Cogent Communications
D	199.7.91.13	University of Maryland
E	192.203.230.10	NASA Ames Research Center
F	192.5.5.241	Internet Systems Consortium
G	192.112.36.4	U.S. Department of Defense (NIC)
H	198.97.190.53	U.S. Army Research Lab
I	192.36.148.17	Netnod
J	192.58.128.30	Verisign, Inc.
K	193.0.14.129	RIPE NCC
L	199.7.83.42	ICANN
M	202.12.27.33	WIDE Project

DNS servers learn how to reach the root servers by consulting a *root hints* file that's located on the server. In the Unix/Linux world, this file is known as `named.root` and can be found at `/etc/named.root`. For Windows, the root hints are stored within Active Directory rather than as a separate file. Listing 6-3 shows a typical `named.root` file.

LISTING 6-3: The named.root File

```
;      This file holds the information on root name servers needed to
      initialize cache of
;      internet domain name servers (for example, reference this file in the
;      "cache . <file>" configuration file of BIND domain name servers).
;
;      This file is made available by InterNIC under anonymous FTP as
;      file                /domain/named.cache
;      on server            FTP.INTERNIC.NET
;      -OR-                 RS.INTERNIC.NET
;
;      last update:         August 12, 2020
;      related version of root zone:      2020081201
;
; FORMERLY NS.INTERNIC.NET
;
.                3600000      NS      A.ROOT-SERVERS.NET.
A.ROOT-SERVERS.NET. 3600000      A      198.41.0.4
A.ROOT-SERVERS.NET. 3600000      AAAA   2001:503:ba3e::2:30
;
; FORMERLY NS1.ISI.EDU
;
.                3600000      NS      B.ROOT-SERVERS.NET.
B.ROOT-SERVERS.NET. 3600000      A      199.9.14.201
B.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:200::b
;
; FORMERLY C.PSI.NET
;
.                3600000      NS      C.ROOT-SERVERS.NET.
C.ROOT-SERVERS.NET. 3600000      A      192.33.4.12
C.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:2::c
;
; FORMERLY TERP.UMD.EDU
;
.                3600000      NS      D.ROOT-SERVERS.NET.
D.ROOT-SERVERS.NET. 3600000      A      199.7.91.13
D.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:2d::d
;
```



```

; FORMERLY NS.NASA.GOV
;
.                3600000      NS      E.ROOT-SERVERS.NET.
E.ROOT-SERVERS.NET. 3600000      A      192.203.230.10
E.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:a8::e
;
; FORMERLY NS.ISC.ORG
;
.                3600000      NS      F.ROOT-SERVERS.NET.
F.ROOT-SERVERS.NET. 3600000      A      192.5.5.241
F.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:2f::f
;
; FORMERLY NS.NIC.DDN.MIL
;
.                3600000      NS      G.ROOT-SERVERS.NET.
G.ROOT-SERVERS.NET. 3600000      A      192.112.36.4
G.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:12::d0d
;
; FORMERLY AOS.ARL.ARMY.MIL
;
.                3600000      NS      H.ROOT-SERVERS.NET.
H.ROOT-SERVERS.NET. 3600000      A      198.97.190.53
H.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:1::53
;
; FORMERLY NIC.NORDU.NET
;
.                3600000      NS      I.ROOT-SERVERS.NET.
I.ROOT-SERVERS.NET. 3600000      A      192.36.148.17
I.ROOT-SERVERS.NET. 3600000      AAAA   2001:7fe::53
;
; OPERATED BY VERISIGN, INC.
;
.                3600000      NS      J.ROOT-SERVERS.NET.
J.ROOT-SERVERS.NET. 3600000      A      192.58.128.30
J.ROOT-SERVERS.NET. 3600000      AAAA   2001:503:c27::2:30
;
; OPERATED BY RIPE NCC
;
.                3600000      NS      K.ROOT-SERVERS.NET.
K.ROOT-SERVERS.NET. 3600000      A      193.0.14.129
K.ROOT-SERVERS.NET. 3600000      AAAA   2001:7fd::1
;
; OPERATED BY ICANN
;
.                3600000      NS      L.ROOT-SERVERS.NET.
L.ROOT-SERVERS.NET. 3600000      A      199.7.83.42
L.ROOT-SERVERS.NET. 3600000      AAAA   2001:500:9f::42
;

```

(continued)

```

; OPERATED BY WIDE
;
.           3600000      NS      M.ROOT-SERVERS.NET.
M.ROOT-SERVERS.NET. 3600000      A       202.12.27.33
M.ROOT-SERVERS.NET. 3600000      AAAA    2001:dc3::35
; End of file

```

Caching

DNS servers don't really like doing all that work to resolve DNS names, but they're not stupid. They know that if a user visits `www.wiley.com` today, he'll probably do it again tomorrow. As a result, name servers keep a cache of query results. The next time the user visits `www.wiley.com`, the name server is able to resolve this name without having to query all those other name servers.

The internet is constantly changing, however, so cached data can quickly become obsolete. For example, suppose that John Wiley & Sons, Inc., switches its website to a different server. It can update its name servers to reflect the new IP address, but any name servers that have a cached copy of the query will be out of date.

To prevent this from being a major problem, DNS data is given a relatively short expiration time. The expiration value for DNS data is called the *TTL* (Time to Live). TTL is specified in seconds. Thus, a TTL of 60 means the data is kept for one minute.

Understanding DNS Queries

When a DNS client needs to resolve a DNS name to an IP address, it uses a library routine — a *resolver* — to handle the query. The resolver takes care of sending the query message over the network to the DNS server, receiving and interpreting the response, and informing the client of the results of the query.

A DNS client can make two basic types of queries: recursive and iterative. The following list describes the difference between these two query types. (The following discussion assumes that the client is asking the server for the IP address of a host name, which is the most common type of DNS query. You find out about other types of queries later; they, too, can be either recursive or iterative.)

- » **Recursive queries:** When a client issues a *recursive DNS query*, the server must reply with either the IP address of the requested host name or an error message indicating that the host name doesn't exist. If the server doesn't have the information, it asks another DNS server for the IP address. When the first server finally gets the IP address, it sends it back to the client. If the server determines that the information doesn't exist, it returns an error message.
- » **Iterative queries:** When a server receives an iterative query, it returns the IP address of the requested host name if it knows the address. If the server doesn't know the address, it returns a *referral*, which is simply the address of a DNS server that should know. The client can then issue an iterative query to the server to which it was referred.

Normally, DNS clients issue recursive queries to DNS servers. If the server knows the answer to the query, it replies directly to the client. If not, the server issues an iterative query to a DNS server that it thinks should know the answer. If the original server gets an answer from the second server, it returns the answer to the client. If the original server gets a referral to a third server, the original server issues an iterative query to the third server. The original server keeps issuing iterative queries until it either gets the answer or an error occurs. It then returns the answer or the error to the client.

Confused? I can understand why. An example may help to clear things up. Suppose that a user wants to view the web page `www.wiley.com`. The following sequence of steps occurs to resolve this address:

1. **The browser asks the client computer's resolver to find the IP address of `www.wiley.com`.**
2. **The resolver issues a recursive DNS query to its name server.**
In this case, I'll call the name server `ns1.LoweWriter.com`.
3. **The name server `ns1.LoweWriter.com` checks whether it knows the IP address of `www.wiley.com`.**
It doesn't, so the name server issues an iterative query to one of the root name servers to see whether it knows the IP address of `www.wiley.com`.
4. **The root name server doesn't know the IP address of `www.wiley.com`, so it returns a list of the name servers that are authoritative for the `com` domain.**

5. The `ns1.LoweWriter.com` name server picks one of the `com` domain name servers and sends it an iterative query for `www.wiley.com`.
6. The `com` name server doesn't know the IP address of `www.wiley.com`, so it returns a list of the name servers that are authoritative for the `wiley.com` domain.
7. The `ns1.LoweWriter.com` name server picks one of the name servers for the `wiley.com` domain and sends it an iterative query for `www.wiley.com`.
8. The `wiley.com` name server knows the IP address for `www.wiley.com`, so the name server returns it.
9. The `ns1.LoweWriter.com` name server shouts with joy for having finally found the IP address for `www.wiley.com`. It gleefully returns this address to the client. It also caches the answer so that the next time the user looks for `www.wiley.com`, the name server won't have to contact other name servers to resolve the name.
10. The client also caches the results of the query.

The next time the client needs to look for `www.wiley.com`, the client can resolve the name without troubling the name server.

Zone Files and Resource Records

Each DNS zone is defined by a *zone file* (also known as a *DNS database* or a *master file*). For Windows DNS servers, the name of the zone file is *domain.zone*. For example, the zone file for the `LoweWriter.com` zone is named `LoweWriter.com.zone`. For BIND DNS servers, the zone files are named *db.domain*. Thus, the zone file for the `LoweWriter.com` domain would be `db.LoweWriter.com`. The format of the zone file contents is the same for both systems, however.

A zone file consists of one or more resource records. Creating and updating the resource records that comprise the zone files is one of the primary tasks of a DNS administrator. The Windows DNS server provides a friendly graphical interface to the resource records. However, you should still be familiar with how to construct resource records.

Resource records are written as simple text lines, with the following fields:

Owner	TTL	Class	Type	RDATA
-------	-----	-------	------	-------

These fields must be separated from each other by one or more spaces. The following list describes the five resource record fields:



TIP

» **Owner:** The name of the DNS domain or the host that the record applies to. This is usually specified as a fully qualified domain name (with a trailing dot) or as a simple host name (without a trailing dot), which is then interpreted in the context of the current domain.

You can also specify a single at symbol (@) as the owner name. In that case, the current domain is used.

» **TTL:** Also known as *Time to Live*; the number of seconds that the record should be retained in a server's cache before it's invalidated. If you omit the TTL value for a resource record, a default TTL is obtained from the Start of Authority (SOA) record.

» **Class:** Defines the protocol to which the record applies. You should always specify IN, for the Internet Protocol. If you omit the class field, the last class field that you specified explicitly is used. As a result, you'll sometimes see zone files that specify IN only on the first resource record (which must be an SOA record) and then allow it to default to IN on all subsequent records.

» **Type:** The resource record type. The most commonly used resource types are summarized in Table 6-6 and are described separately later in this section. Like the Class field, you can also omit the Type field and allow it to default to the last specified value.

» **RDATA:** Resource record data that is specific to each record type.

TABLE 6-6

Common Resource Record Types

Type	Name	Description
SOA	Start of Authority	Identifies a zone
NS	Name Server	Identifies a name server that is authoritative for the zone
A	Address	Maps a fully qualified domain name to an IP address
CNAME	Canonical Name	Creates an alias for a fully qualified domain name
MX	Mail Exchange	Identifies the mail server for a domain
PTR	Pointer	Maps an IP address to a fully qualified domain name for reverse lookups



REMEMBER

Most resource records fit on one line. If a record requires more than one line, you must enclose the data that spans multiple lines in parentheses.



TIP

You can include comments to clarify the details of a zone file. A comment begins with a semicolon (;) and continues to the end of the line. If a line begins with a semicolon, the entire line is a comment. You can also add a comment to the end of a resource record. You see examples of both types of comments later in this chapter.

SOA records

Every zone must begin with an SOA record, which names the zone and provides default information for the zone. Table 6-7 lists the fields that appear in the RDATA section of an SOA record. Note that these fields are positional, so you should include a value for all of them and list them in the order specified. Because the SOA record has so many RDATA fields, you'll probably need to use parentheses to continue the SOA record onto multiple lines.

TABLE 6-7 RDATA Fields for an SOA Record

Name	Description
MNAME	The domain name of the name server that is authoritative for the zone.
RNAME	An email address (specified in domain name format; not regular email format) of the person responsible for this zone.
SERIAL	The serial number of the zone. Secondary zones use this value to determine whether they need to initiate a zone transfer to update their copy of the zone.
REFRESH	A time interval that specifies how often a secondary server should check whether the zone needs to be refreshed. A typical value is 3600 (one hour).
RETRY	A time interval that specifies how long a secondary server should wait after requesting a zone transfer before trying again. A typical value is 600 (ten minutes).
EXPIRE	A time interval that specifies how long a secondary server should keep the zone data before discarding it. A typical value is 86400 (one day).
MINIMUM	A time interval that specifies the TTL value to use for zone resource records that omit the TTL field. A typical value is 3600 (one hour).

Note two things about the SOA fields:

» The email address of the person responsible for the zone is given in DNS format, not in normal email format. Thus, you separate the user from the

mail domain with a dot rather than an at symbol (@). For example, `doug@Lowewriter.com` would be listed as `doug.lowewriter.com`.

» **The serial number should be incremented every time you change the zone file.** If you edit the file via the graphic interface provided by Windows DNS, the serial number is incremented automatically. However, if you edit the zone file via a simple text editor, you have to manually increment the serial number.

Here's a typical example of an SOA record, with judicious comments to identify each field:

```
lowewriter.com. IN SOA (  
  ns1.lowewriter.com ; authoritative name server  
  doug.lowewriter.com ; responsible person  
  148 ; version number  
  3600 ; refresh (1 hour)  
  600 ; retry (10 minutes)  
  86400 ; expire (1 day)  
  3600 ) ; minimum TTL (1 hour)
```

NS records

Name server (NS) records identify the name servers that are authoritative for the zone. Every zone must have at least one NS record. Using two or more NS records is better so that if the first name server is unavailable, the zone will still be accessible.

The owner field should either be the fully qualified domain name for the zone, with a trailing dot, or an at symbol (@). The RDATA consists of just one field: the fully qualified domain name of the name server.

The following examples show two NS records that serve the `lowewriter.com` domain:

```
lowewriter.com. IN NS ns1.lowewriter.com.  
lowewriter.com. IN NS ns2.lowewriter.com.
```

A records

Address (A) records are the meat of the zone file: They provide the IP addresses for each of the hosts that you want to make accessible via DNS. In an A record, you usually list just the host name in the owner field, thus allowing DNS to add the

domain name to derive the fully qualified domain name for the host. The RDATA field for the A record is the IP address of the host.

The following lines define various hosts for the `Lowewriter.com` domain:

```
doug IN A 192.168.168.200
server1 IN A 192.168.168.201
debbie IN A 192.168.168.202
printer1 IN A 192.168.168.203
router1 IN A 207.126.127.129
www IN A 64.71.129.102
```

Notice that for these lines, I don't specify the fully qualified domain names for each host. Instead, I just provide the host name. DNS will add the name of the zone's domain to these host names in order to create the fully qualified domain names.

If I wanted to be more explicit, I could list these A records like this:

```
doug.lowewriter.com. IN A 192.168.168.200
server1.lowewriter.com. IN A 192.168.168.201
debbie.lowewriter.com. IN A 192.168.168.202
printer1.lowewriter.com. IN A 192.168.168.203
router1.lowewriter.com IN A 207.126.127.129
www.lowewriter.com. IN A 64.71.129.102
```

However, all this does is increase the chance for error. Plus, it creates more work for you later if you decide to change your network's domain.

CNAME records

A *Canonical Name* (CNAME) record creates an alias for a fully qualified domain name. When a user attempts to access a domain name that is actually an alias, the DNS system substitutes the real domain name — known as the *Canonical Name* — for the alias. The owner field in the CNAME record provides the name of the alias that you want to create. Then, the RDATA field provides the Canonical Name — that is, the real name of the host.

For example, consider these resource records:

```
ftp.lowewriter.com. IN A 207.126.127.132
files.lowewriter.com. IN CNAME www1.lowewriter.com.
```


Here, the host name of an FTP server at 207.126.127.132 is `ftp.lowewriter.com`. The CNAME record allows users to access this host as `files.lowewriter.com` if they prefer.

PTR records

A *Pointer* (PTR) record is the opposite of an address record: It provides the fully qualified domain name for a given address. The owner field should specify the reverse lookup domain name, and the RDATA field specifies the fully qualified domain name. For example, the following record maps the address 64.71.129.102 to `www.lowewriter.com`:

```
102.129.71.64.in-addr.arpa. IN PTR www.lowewriter.com.
```

PTR records don't usually appear in normal domain zones. Instead, they appear in special reverse lookup zones. For more information, see the section "Reverse Lookup Zones," later in this chapter.

MX records

Mail Exchange (MX) records identify the mail server for a domain. The owner field provides the domain name that users address mail to. The RDATA section of the record has two fields. The first is a priority number used to determine which mail servers to use when several are available. The second is the fully qualified domain name of the mail server itself.

For example, consider the following MX records:

```
lowewriter.com. IN MX 0 mail1.lowewriter.com.  
lowewriter.com. IN MX 10 mail2.lowewriter.com.
```

In this example, the `lowewriter.com` domain has two mail servers, named `mail1.lowewriter.com` and `mail2.lowewriter.com`. The priority numbers for these servers are 0 and 10. Because it has a lower priority number, mail will be delivered to `mail1.lowewriter.com` first. The `mail2.lowewriter.com` server will be used only if `mail1.lowewriter.com` isn't available.



TIP

The server name specified in the RDATA section should be an actual host name, not an alias created by a CNAME record. Although some mail servers can handle MX records that point to CNAMEs, not all can. As a result, you shouldn't specify an alias in an MX record.



Be sure to create a reverse lookup record (PTR, described in the next section) for your mail servers. Some mail servers won't accept mail from a server that doesn't have valid reverse lookup entries.

Reverse Lookup Zones

Normal DNS queries ask a name server to provide the IP address that corresponds to a fully qualified domain name. This kind of query is a *forward lookup*. A *reverse lookup* is the opposite of a forward lookup: It returns the fully qualified domain name of a host based on its IP address.

Reverse lookups are possible because of a special domain called the `in-addr.arpa` domain, which provides a separate fully qualified domain name for every possible IP address on the internet. To enable a reverse lookup for a particular IP address, all you have to do is create a PTR record in a reverse lookup zone (a zone that is authoritative for a portion of the `in-addr.arpa` domain). The PTR record maps the `in-addr.arpa` domain name for the address to the host's actual domain name.

The technique used to create the reverse domain name for a given IP address is pretty clever. It creates subdomains beneath the `in-addr.arpa` domain by using the octets of the IP address, listing them in reverse order. For example, the reverse domain name for the IP address 207.126.67.129 is `129.67.126.207.in-addr.arpa`.

Why list the octets in reverse order? Because that correlates the network portions of the IP address (which work from left to right) with the subdomain structure of DNS names (which works from right to left). The following description should clear this up:

- » The 255 possible values for the first octet of an IP address each have a subdomain beneath the `in-addr.arpa` domain. For example, any IP address that begins with 207 can be found in the `207.in-addr.arpa` domain.
- » Within this domain, each of the possible values for the second octet can be found as a subdomain of the first octet's domain. Thus, any address that begins with 207.126 can be found in the `126.207.in-addr.arpa` domain.
- » The same holds true for the third octet, so any address that begins with 207.126.67 can be found in the `67.126.207.in-addr.arpa` domain.
- » By the time you get to the fourth octet, you've pinpointed a specific host. The fourth octet completes the fully qualified reverse domain name. Thus, 207.126.67.129 is mapped to `129.67.126.207.in-addr.arpa`.

As a result, to determine the fully qualified domain name for the computer at 207.126.67.129, the client queries its DNS server for the FQDN that corresponds to 129.67.126.207.in-addr.arpa.

Working with the Windows DNS Server

Installing and managing a DNS server depends on the network operating system (NOS) that you're using. The following sections are specific to working with a DNS server in Windows Server 2025. Working with BIND in a Unix/Linux environment is similar but without the help of a graphical user interface (GUI), and working with Windows Server 2016 is similar.

You can install the DNS server on a Windows server from the Server Manager application. Open the Server Manager and choose Manage ➤ Add Roles and Features. Then, follow the wizard's instructions to add the DNS Role.

After you set up a DNS server, you can manage the DNS server from the DNS Manager, as shown in Figure 6-3. From this management console, you can perform common administrative tasks, such as adding additional zones, changing zone settings, adding A or MX records to an existing zone, and so on. The DNS Manager hides the details of the actual resource records from you, thus allowing you to work with a friendly GUI instead.

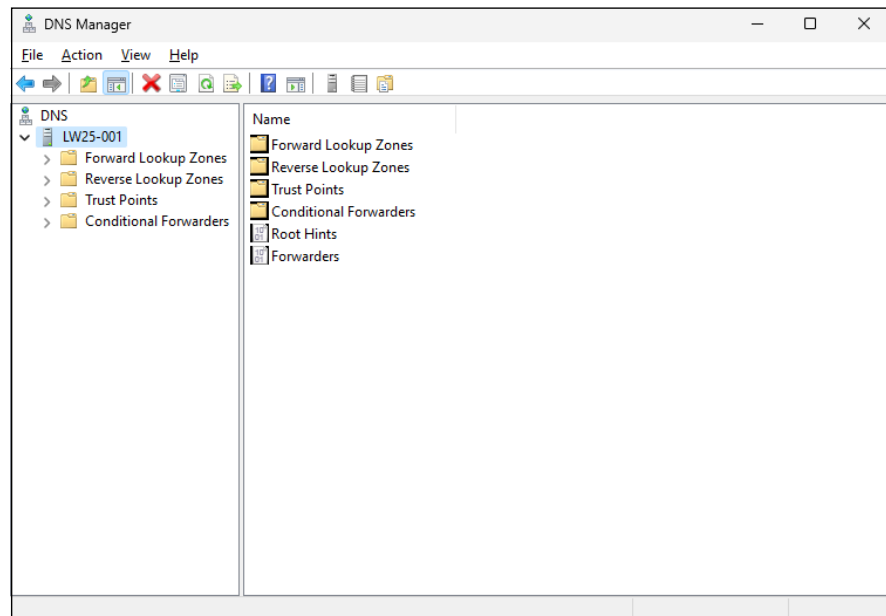


FIGURE 6-3:
The DNS
Manager.

Creating a new zone

To add a zone to be managed by a DNS server, follow these steps:

1. Right-click Forward Lookup Zones and choose New Zone.

The New Zone Wizard, shown in Figure 6-4, appears.

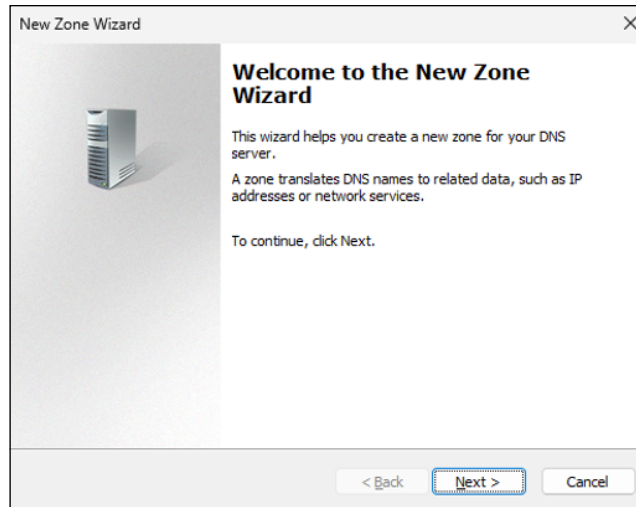


FIGURE 6-4:
The New
Zone Wizard
greet you.

2. Click Next.

The wizard starts by asking which type of zone you'd like to create (see Figure 6-5). There are three choices:

- **Primary Zone:** A zone that will be managed directly by this server.
- **Secondary Zone:** A read-only copy of a zone that is managed by another DNS server.
- **Stub Zone:** Creates a copy of a zone with just an NS, SOA, and possible H records. A stub zone is not authoritative for the zone.

For our purposes here, we'll create a primary zone.

3. Select Primary Zone and click Next.

The wizard asks for the name of the zone (see Figure 6-6).

4. Enter the zone name and click Next.

The zone name should be the full DNS name of the zone you want to manage (for example, lowewriter.com).

The wizard offers to create a new zone file, as shown in Figure 6-7.

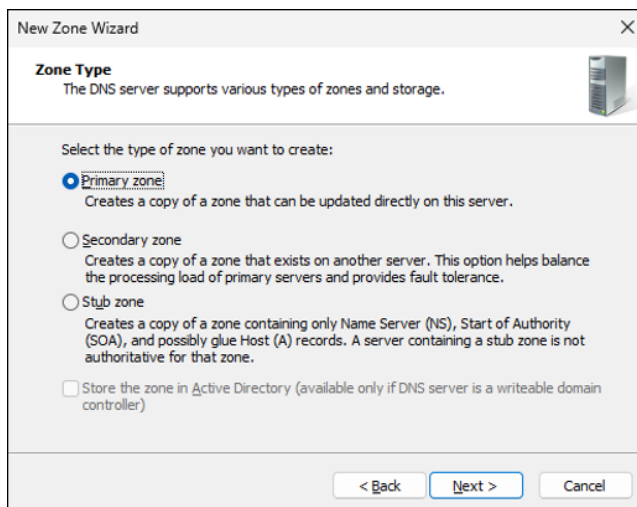


FIGURE 6-5:
The New Zone Wizard asks for the zone type.

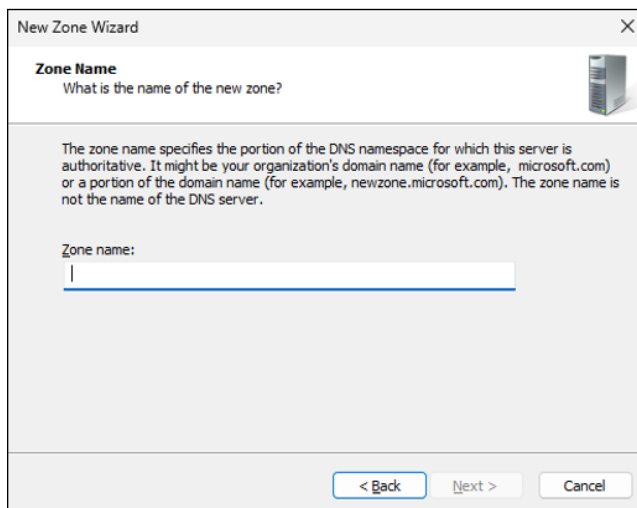


FIGURE 6-6:
The New Zone Wizard asks for the zone name.

5. Click Next to let the wizard create a new zone file.

Optionally, you could import an existing zone file. You must first obtain the zone file from an existing DNS server, and then drop it in the %SystemRoot%\System32\dns folder.

The wizard offers to let you switch from a secure mode in which DNS must be updated manually to a dynamic update mode, which is terribly insecure because it allows any DNS client to update the zone (see Figure 6-8). Don't enable dynamic updates.

FIGURE 6-7:
The New Zone Wizard offers to create a new zone file.

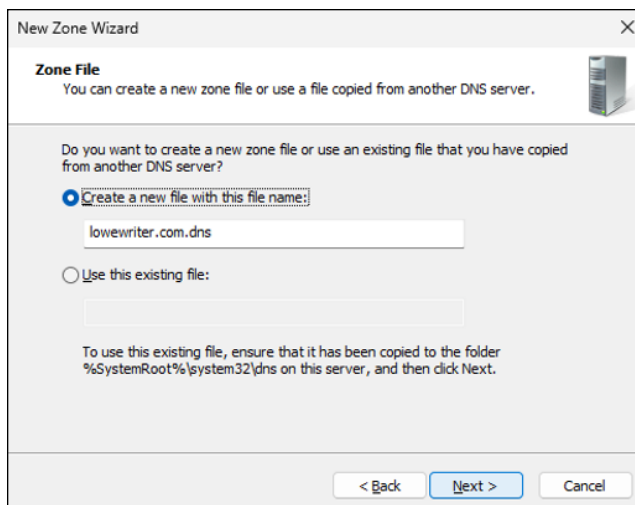
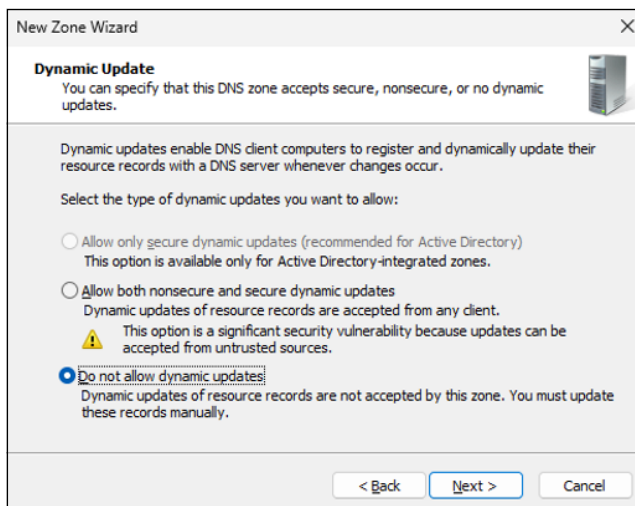


FIGURE 6-8:
The New Zone Wizard asks you how the zone should be updated.



6. Select Do Not Allow Dynamic Updates and click Next.

The wizard displays a summary of all your choices.

7. Click Finish.

Presto! Your new zone now appears in the Forward Lookup Zones section of the DNS Manager, as shown in Figure 6-9.

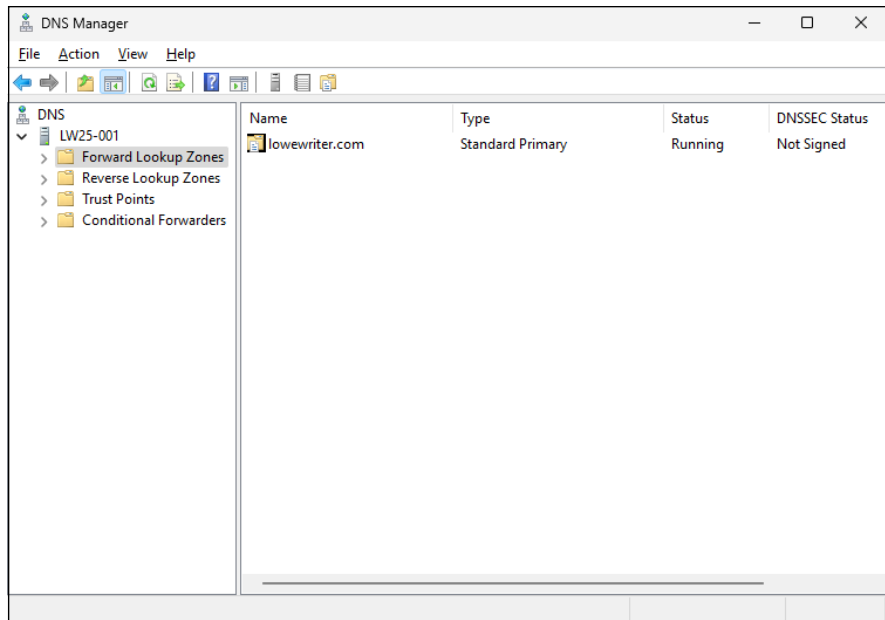


FIGURE 6-9:
Congratulations
on creating your
first DNS zone!

Creating a new host record

To add a new host (that is, an A record) to a zone, right-click the zone in the Forward Management Zones section of the DNS Manager. Then choose the Add New Host command. This brings up the New Host dialog box, shown in Figure 6-10. From this dialog box, specify the following information.

- » **Name:** The host name for the new host.
- » **IP Address:** The host's IP address.
- » **Create Associated Pointer (PTR) Record:** Automatically creates a PTR record in the reverse lookup zone file. Select this option if you want to allow reverse lookups for the host.

You can add other records, such as MX or CNAME records, in the same way.

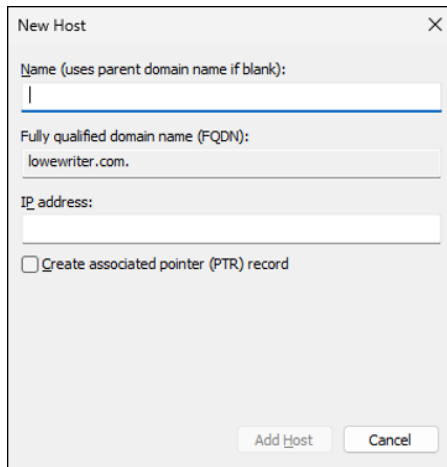


FIGURE 6-10:
The New Host
dialog box.

How to Configure a Windows DNS Client

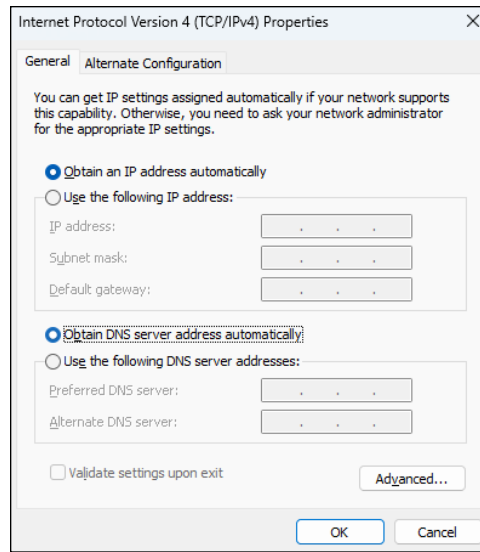
Client computers don't need much configuration in order to work properly with DNS. The client must have the address of at least one DNS server. Usually, this address is supplied by DHCP, so if the client is configured to obtain its IP address from a DHCP server, it will also obtain the DNS server address from DHCP.

If you must configure DHCP manually, follow these steps:

- 1. Open the Control Panel.**
If you haven't already, switch to Small Icons view.
- 2. Open Network and Sharing Center.**
- 3. Click the link for your wired or wireless network adapter.**
The adapter's Status dialog box appears.
- 4. Click Properties.**
The adapter's Properties dialog box appears.
- 5. Select Internet Protocol Version 4 and then click Properties.**

The Internet Protocol Version 4 (TCP/IPv4) Properties dialog box, shown in Figure 6-11, appears.

FIGURE 6-11:
Configuring a
Windows client
to obtain its DNS
address from
DHCP.



6. Configure the DNS options.

To pull DNS from the DHCP server, choose Obtain DNS Server Address Automatically.

To manually specify the DNS servers, choose Use the Following DNS Server Addresses, and then enter the IP addresses for your preferred and secondary DNS servers.

7. Click OK several times until all the dialog boxes you've opened are gone.

